

Title: AOSP (Android open-source project)

Discription:

Building an entire android OS, basic structure of android including the Linux kernel, core libraries, and the Android runtime without any vendor specific applications that can be used to flash on any board like raspberry pi or any SOC (system on chip).

Applications:

Used by different vendors to build their own customized OS containing their own applications and configurations.

Project:

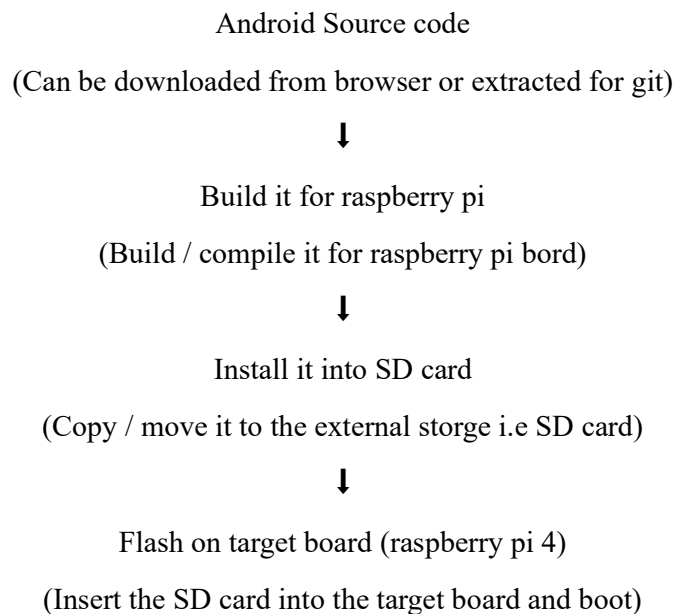
Development machine requirement:

1. Ubuntu PC
2. 32GB RAM Recommended
3. 500GB SSD
4. Multi-Core CPU

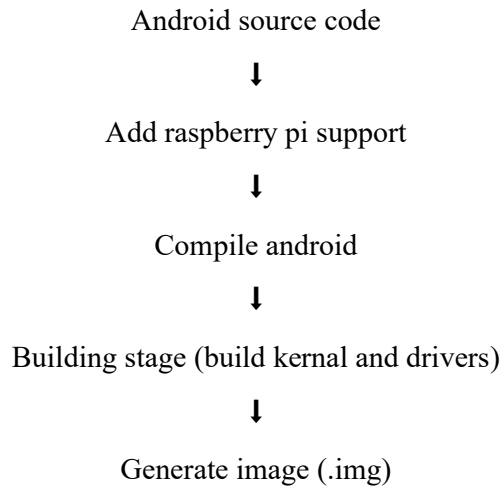
Target machine:

Raspberry pi 4 (soc)

Basic understanding flow:



Flow including steps in building:



Project steps by step working:

Step 1: Get android source code

Create a folder for the project naming AOSP for easy identification.

open the terminal and change the directory to AOSP using cd command

now use git commands to create. git hidden folder which is done using git init

once done using the clone command extract the android of required version

```
repo init -u https://android.googlesource.com/platform/manifest \-b android-15.0.0_r32  
curl -o .repo/local_manifests/manifests_brcm_rpi.xml -L https://githubusercontent.com  
\-create-dires
```

```
repo sync -c -j$(nproc)
```

check for the files present in the android using \$ls

Once the android source code is cloned or sync then the directory will be populated with

Aosp/	— frameworks:	contains code for managers
	— packages:	contains applications
	— system:	core android services
	— hardware:	Hardware interface codes
	— kernel:	linux kernal code
	— vendor:	vendor specific source
	— device:	board configuration

Note: Out file will be generated once the build is completed.

Step 2: install build dependencies

```
sudo apt install git-core gnupg flex bison build-essential zip curl \
zlib1g-dev gcc-multilib g++-multilib libc6-dev-i386 \
lib32ncurses5-dev x11proto-core-dev libx11-dev \
lib32z1-dev libgl1-mesa-dev libxml2-utils xsltproc unzip
```

These dependencies are used to build the android using the requirements like

Compiler, Linker, Libraries, Java support, Build tools

Fetch android 15 compatible linux kernel

```
Mkdir ~/rpi_kernal && cd
```

```
Repo init -u https://android.googlesource.com/kernelmanifest -b comman-on-android15-6.6-lts
```

```
curl -o .repo/local_manifests/manifests_brcm_rpi.xml -L https://githubusercontent.com -create-dires
```

```
repo sync -c -j$(nproc)
```

```
build/build.sh
```

this will download matching linux tree to support android on raspberry pi

Step 3: configure build

In this step we say / inform the android to build image for raspberry pi, to do so setup an environment and start build

```
source build/envsetup.sh
```

with this the android will load the build functions into the terminal that are required to build

like lunch, m, mm, mmm, simply loads android build toolkit.

then use the command lunch

lunch is a build configuration utility used for setting target device.

```
lunch aosp_rpi4-userdebug
```

with this the android will initialize the variables with

```
TARGET_PRODUCT=rpi4
```

```
TARGET_ARCH=arm64
```

```
TARGET_BUILD_VARIANT=user-debug.
```

Step 4: Start build using m / mm / mmm

Command used to start the compilation and build: **m -jN**

M used to Compile the entire Android system from the top of the tree, N refers to number of threads

Android starts compiling thousands of files like java, c, c++, xml.

This step is crucial and requires hours of time, here android uses build-system - “SOONG”

Soong creates the ninja files, ninja files are low-level, machine-generated instruction that act as the final blueprint for compiling your software.

Simply we can say soong creates a blueprint (.ninja files) and gives it to ninja

Source code → soong → build.ninja → ninja → images

This .ninja file consists of many things like Compile Settings.java, Compile Launcher.java, Compile frameworks/base, Compile libc, Compile kernel, Link libraries, Create APKs, Create Images.

Ninja analyses the .ninja files which are like a blueprint that are used to build the images

Ninja first builds the dependencies and then the actual images for example

Settings.img needs framework.jar and framework.jar needs libc

Ninja uses the respective compiler which are downloaded during the step 2 (dependencies) and compiles the files like .c .cpp .java etc

Also compile libraries like libcam libc etc

Step 5: linker

Links multiple “.o” files like main.o wifi.o audio.o using the linker like lld linker and generates executable files.

Step 6: framework building

Android builds the “framework/base” folder or directory the ninja will compile this file as well and creates the .jar files like framework.jar and services.jar

These .jar contains ActivityManager, PackageManager, WindowManager,

these are the java-based background services that are invoked once the machine boots.

Step 7: Building applications and other utilities

Android starts building

Built-in applications: Settings Launcher3 Dialer SystemUI etc

Build APEX: Runtime Media Neural Networks

Builds kernel: to run the os in raspberry kernel is essential

Builds HAL: Hardware Abstraction Layer is a framework passes requests to the hardware from HAL module
(photo click request sent to module then HAL passes it to camera)

Vendor applications: vendor applications

Step 8: Package system

Android collects everything that it just built like

Framework, Apps, Libraries, HALs, APEX, Vendor Files.

Step 9: generate image

Now android creates the .img file which are termed as images like

Boot.img: used to for booting

System.img: used for internal applications frameworks (largest partition)

Vendor.img: contains the vendor components

Userdata.img: contains basic utility components like settings built-in apps

Step 10: loading image files to external storage and mount into target

Copy/ move / load the image files into the external storage like SD card and insert the SD

Into the target.

Step 11: boot target

Boot / power on the target (raspberry pi) then the target will be flashed with the OS and display the termina or desktop (android icon).

Images of process while building:

```
semicorelabs@semicorelabs-OptiPlex-7060:~$ sudo apt install -y \
git git-lfs curl wget \
openjdk-17-jdk \
python3 python-is-python3 \
flex bison gperf build-essential \
zip unzip \
zlib1g-dev \
gcc-multilib g++-multilib \
libc6-dev-i386 \
lib32z1-dev \
libncurses-dev \
x11proto-core-dev \
libx11-dev \
libgl1-mesa-dev \
libxml2-utils \
xsltproc \
```

Downloading dependencies

```
semicorelabs@semicorelabs-OptiPlex-7060:~/aosp$ source build/envsetup.sh
semicorelabs@semicorelabs-OptiPlex-7060:~/aosp$ lunch

You're building on Linux
Lunch menu .. Here are the common combinations:
1. aosp_arm-eng
2. aosp_arm64-eng
3. aosp_barbet-userdebug
4. aosp_bluejay-userdebug
5. aosp_bluejay_car-userdebug
6. aosp_bramble-userdebug
7. aosp_bramble_car-userdebug
8. aosp_car_arm-userdebug
9. aosp_car_arm64-userdebug
10. aosp_car_x86-userdebug
11. aosp_car_x86_64-userdebug
12. aosp_cf_arm64_auto-userdebug
13. aosp_cf_arm64_phone-userdebug
14. aosp_cf_x86_64_foldable-userdebug
15. aosp_cf_x86_64_only_phone_hsum-userdebug
16. aosp_cf_x86_64_pc-userdebug
17. aosp_cf_x86_64_phone-userdebug
18. aosp_cf_x86_64_tv-userdebug
19. aosp_cf_x86_auto-userdebug
20. aosp_cf_x86_phone-userdebug
21. aosp_cf_x86_tv-userdebug
22. aosp_cheetah-userdebug
23. aosp_cloudrtpsr-userdebug
24. aosp_coral-userdebug
25. aosp_coral_car-userdebug
26. aosp_felix-userdebug
27. aosp_flame-userdebug
28. aosp_flame_car-userdebug
29. aosp_lynx-userdebug
30. aosp_oriole-userdebug
31. aosp_oriole_car-userdebug
32. aosp_panther-userdebug
33. aosp_raven-userdebug
34. aosp_raven_car-userdebug
35. aosp_ravencat-userdebug
36. aosp_redfin-userdebug
37. aosp_redfin_car-userdebug
38. aosp_redfin_vf-userdebug
39. aosp_slider-userdebug
```

Selecting the target (lunch phase)

```
build/nake/core/soong_config.mk:209: warning: BOARD_PLAT_PUBLIC_SEPOLICY_DIR has been depre
cated. Use SYSTEM_EXT_PUBLIC_SEPOLICY_DIRS instead.
build/nake/core/soong_config.mk:210: warning: BOARD_PLAT_PRIVATE_SEPOLICY_DIR has been depr
ecated. Use SYSTEM_EXT_PRIVATE_SEPOLICY_DIRS instead.
=====
PLATFORM_VERSION_CODENAME=REL
PLATFORM_VERSION=13
TARGET_PRODUCT=aosp_cf_x86_64_phone
TARGET_BUILD_VARIANT=userdebug
TARGET_BUILD_TYPE=release
TARGET_ARCH=x86_64
TARGET_ARCH_VARIANT=silvermont
TARGET_2ND_ARCH=x86
TARGET_2ND_ARCH_VARIANT=silvermont
HOST_ARCH=x86_64
HOST_2ND_ARCH=x86
HOST_OS=linux
HOST_OS_EXTRA=Linux-6.17.0-35-generic-x86_64-Ubuntu-24.04.4-LTS
HOST_CROSS_OS=windows
HOST_CROSS_ARCH=x86
HOST_CROSS_2ND_ARCH=x86_64
HOST_BUILD_TYPE=release
BUILD_ID=TQ3A.230901.001
OUT_DIR=out
PRODUCT_SOONG_NAMESPACES=device/generic/goldfish-opengl device/generic/goldfish device/gene
ric/goldfish-opengl hardware/google/camera hardware/google/camera/devices/EmulatedCamera de
vice/google/cuttlefish/apex/com.google.cf.wifi_hwsim external/mesa3d vendor/google_devices/
common/proprietary/confirmationui_hal
0:48 analyzing Android.bp files and generating ninja file at out/soong/build.ninja
[ 91% 163/179] including system/sepolicy/Android.mk ...
system/sepolicy/Android.mk:57: warning: BOARD_PLAT_PUBLIC_SEPOLICY_DIR has been deprecated. Use SYSTEM_EXT_PUBLIC_SEPOLICY_DIRS instead.
system/sepolicy/Android.mk:62: warning: BOARD_PLAT_PRIVATE_SEPOLICY_DIR has been deprecated
[ 5% 7698/144023] build out/target/product/vsoc_x86_64/system/etc/event-log-tags
packages/apps/Settings/src/com/android/tv/settings/EventLogTags.logtags:6: warni
ng: tag "lock_screen_type" (90200) duplicated in packages/apps/Settings/src/com/android/set
tings/EventLogTags.logtags:6
packages/apps/TvSettings/Settings/src/com/android/tv/settings/EventLogTags.logtags:9: warni
ng: tag "exp_det_device_admin_activated_by_user" (90201) duplicated in packages/apps/Settin
gs/src/com/android/settings/EventLogTags.logtags:9
packages/apps/TvSettings/Settings/src/com/android/tv/settings/EventLogTags.logtags:12: warn
ing: tag "exp_det_device_admin_declined_by_user" (90202) duplicated in packages/apps/Settin
gs/src/com/android/settings/EventLogTags.logtags:12
packages/apps/TvSettings/Settings/src/com/android/tv/settings/EventLogTags.logtags:15: warn
ing: tag "exp_det_device_admin_uninstalled_by_user" (90203) duplicated in packages/apps/Set
tings/src/com/android/settings/EventLogTags.logtags:15
0:03 writing build rules ...
```

Building phase

